

شرح مشروع NICE RAG من البداية

من اختيار Guideline سرطان القولون والمستقيم إلى البحث الدلالي وتجهيز نظام الإجابة بالمصادر

NICE NG151

Colorectal Cancer Guideline

هذا الملف يشرح ماذا فعلنا، لماذا فعلناه، ما الملفات الناتجة، وكيف تتحول أسئلة المستخدم إلى نتائج موثقة من الـ Guideline.

نسخة تعليمية مبسطة - مناسبة للمراجعة والعرض في الهاكاثون

الصورة الكبيرة

0

هدف المشروع هو بناء نظام طبي يجيب عن الأسئلة باستخدام Guideline موثوق، بدل الاعتماد على ذاكرة النموذج أو مقالات عشوائية. كل إجابة في النهاية يجب أن يمكن تتبعها إلى التوصية والصفحة والمصدر.

الفكرة في سطر واحد

نحول الـ PDF إلى أجزاء صغيرة منظمة، نحول معنى كل جزء إلى Vector، نخزنه، ثم نبحث عن أقرب أجزاء لسؤال المستخدم ونستخدم ونستخدمها في تكوين الإجابة.



1 اختيار فكرة السرطان والـ Guideline

اخترنا سرطان القولون والمستقيم - Colorectal Cancer - واستخدمنا NICE NG151 كمرجع أساسي. NICE مصدر رسمي، والتوصيات فيه مرقمة ومنظمة إلى Sections و Subsections، لذلك يناسب الاستخراج والبحث وإظهار الـ Citation.

- مصدر طبي رسمي وموثوق بدل صفحات إنترنت غير معروفة.
- التوصيات مرقمة مثل 1.3.1 و 1.6.1.
- الصفحات والجداول واضحة ويمكن الرجوع إليها.
- يمكن إظهار رقم التوصية ورقم الصفحة في الإجابة النهائية.

ليه العمق أفضل من كثرة المصادر؟

في نسخة الهاكاثون الأولى، توثيق Guideline واحد بشكل جيد أفضل من جمع مصادر كثيرة بجودة غير مضمونة. بعد نجاح الـ MVP يمكن إضافة Guidelines أخرى.

2 تنزيل الـ PDF وتسجيل المصدر

نزلنا ملف الـ PDF الرسمي وحفظناه داخل data/raw. كلمة raw تعني النسخة الأصلية كما جاءت من المصدر من غير تعديل، حتى نرجع إليها عند مراجعة أي نص أو جدول أو رقم صفحة.

```
data/raw/nice_ng151_colorectal_cancer.pdf
data/source_manifest.json
```

أنشأنا Source Manifest لتسجيل اسم الوثيقة، كود NG151، المرض، رابط المصدر واسم الملف. هذا يحقق أي معلومة يمكن تتبعها إلى الوثيقة الأصلية: traceability.

3 تحويل الـ PDF إلى Markdown

3

الـ PDF ممتاز للقراءة البشرية لكنه صعب في المعالجة البرمجية. لذلك استخدمنا سكريبت يحول المحتوى إلى منظم، بحيث تظهر العناوين والتوصيات والجداول كنص يمكن فحصه وتقسيمه Markdown.

```
scripts/01_pdf_to_markdown.py
data/processed/nice_ng151_colorectal_cancer.md
```

ليه Markdown؟

لأنه نص بسيط ومنظم، يحافظ على العناوين والقوائم والجداول، ويسهل البحث والمراجعة وبناء الـ Chunks.

4 حفظ الصفحات وفحص الجودة

4

حفظنا كل صفحة كسجل مستقل في ملف JSONL. وجود page_number مع النص ضروري حتى تخرج الإجابة لاحقاً ومعها رقم الصفحة الصحيح.

```
data/processed/nice_ng151_colorectal_cancer_pages.jsonl
```

بعد التحويل شغلنا Quality Assurance للتأكد من أن الصفحات ليست فارغة، وأن التوصيات المرقمة والعناوين والجداول لم تختف أثناء الاستخراج.

قاعدة مهمة

إذا كان النص المستخرج خطأ، فكل ما بعده سيكون خطأ: Chunks خطأ، Embeddings خطأ، ثم إجابات خطأ. لذلك فحص التحويل خطوة أساسية وليست اختيارية.

تقسيم النص إلى Chunks

5

الـ Chunk هو جزء صغير مستقل من الـ Guideline. بدل إرسال الملف كله عند كل سؤال، نبحث عن الأجزاء المرتبطة بالسؤال فقط.

```
scripts/02_build_chunks.py
data/chunks/nice_ng151_colorectal_cancer_chunks.jsonl
```

- كل Recommendation بقي Chunk مستقل حتى لا يتقطع المعنى.
- العنوان ورقم الصفحة ورقم التوصية محفوظون مع النص.
- الجداول مقسمة حسب الصفحة وأجزاؤها مرتبطة ببعض.
- جدول علاج سرطان المستقيم المبكر مرتبط بالتوصية 1.3.3.

لماذا لم نقسم كل 500 كلمة؟

التقسيم العشوائي قد يقطع توصية طبية من المنتصف. استخدمنا Structure-aware Chunking يحترم بنية NICE ويحافظ على الوحدة الدلالية لكل توصية.

MVP في المصدر و 33 للـ 178 Chunk

6

نتج من الوثيقة 178 Chunk تشمل توصيات وجداول ونصوص داعمة. للنسخة الأولى اخترنا 33 Chunk فقط بعلامة `in_initial_scope=true`.

- الـ 33 تشمل الأقسام المتفق عليها والتوصيات المهمة والجداول المرتبطة بها.
- النطاق الصغير يسمح باختبار دقيق وإصلاح الأخطاء بسرعة.
- بعد نجاح الـ MVP يمكن توسيع الفهرس إلى كل التوصيات أو إلى الـ 178 Chunk.

تجهيز النص المناسب للـ Embedding

7

كل Chunk له content وهو النص الأصلي، وله text وهو النص المجهز للبحث. حقل text يضيف اسم الوثيقة والقسم والعنوان والصفحات قبل المحتوى، حتى يفهم الموديل سياق التوصية بشكل أفضل.

```
Document: Colorectal cancer (NG151)
Pages: 26-26
Section: 1.6 Ongoing care and support
Recommendation: 1.6.1
For people who have had potentially curative surgical treatment...
```

ما هو الـ Embedding؟

8

الـ Embedding هو تحويل معنى النص إلى قائمة أرقام. كل Chunk عندنا يتحول إلى Vector طوله 384 رقم. النصوص المتقاربة في المعنى تنتج Vectors متقاربة، حتى لو كان السؤال بالعربي والنص بالإنجليزي.

```
[0.021, -0.047, 0.012, 0.063, ...] -> 384 values
```

مهم: المكتبة ليست الموديل

هي المكتبة التي نشغل بها النماذج. الموديل الذي اخترناه داخلها هو sentence-transformers intfloat/multilingual-e5-small.

اختيار موديل E5 Multilingual 9

اخترنا intfloat/multilingual-e5-small لأنه متعدد اللغات، يفهم العربي والإنجليزي، ومتدرب خصيصا على Retrieval من سؤال إلى فقرة. يعمل محليا ولا يحتاج API key لخدمة Embeddings مدفوعة.

- مناسب لأن الـ Guideline إنجليزي وأسئلة الديمو قد تكون عربية.
- متخصص في query-to-passage retrieval بدل تشابه الجمل فقط.
- يعطي Vectors بطول 384، وهو مناسب لحجم مشروع الهاكاثون.
- يعمل Local ويتم إعادة استخدام الموديل من الـ cache بعد أول تنزيل.

لماذا غيرنا الموديل الأول؟

الموديل الأول العام نجح في 4 من 5 اختبارات، لكنه لم يرجع توصية المتابعة الصحيحة لسؤال عربي ضمن أول 5 نتائج. أعاد التوصية الصحيحة في المركز الأول، فأصبح الاختبار 5 من E5 5.

الفرق بين query و passage 10

تعليمات E5 الرسمية تطلب إضافة prefix داخلي. نستخدم passage قبل النص المخزن و query قبل سؤال المستخدم. هذه الكلمات لا تظهر في واجهة المستخدم، لكنها تساعد الموديل يفهم وظيفة كل نص.

```
passage: Offer follow-up for detection of local recurrence...
query: What follow-up is recommended after curative surgery?
```

11 تخزين الـ Vectors في Chroma

استخدمنا Chroma كـ Vector Database محلية. السكريبت يحسب Embeddings للـ 33 Chunk ثم يخزن النص والـ nice_ng151_colorectal اسمها Collection في Metadata والـ Vector

```
scripts/03_build_vector_index.py
data/vector_store/chroma/
Collection: nice_ng151_colorectal
Vectors: 33 | Dimension: 384 | Metric: cosine
```

ليه Vector Database؟

البحث بالكلمات قد لا يطابق السؤال العربي مع النص الإنجليزي. البحث بالـ Vector يقارن المعنى، لذلك يفهم أن المتابعة قريبة من Follow-up وأن الجراحة قريبة من Surgical treatment.

12 الـ Metadata والـ Citations

الـ Vector يجد النص المناسب، لكن الـ Metadata هي التي تخبرنا من أي وثيقة وقسم وصفحة وتوصية جاء النص. خزنا الحقول المطلوبة في العرض وأضفنا حقولا أخرى مفيدة.

- اسم الوثيقة - document_name
- عنوان القسم - section_title
- الصفحات - page_start و page_end و page_number
- هوية الجزء والتوصية - recommendation_ids و chunk_id
- الرسمي NICE رابط - source_url
- نوع المحتوى وروابط الجداول - linked_chunk_ids و content_type

13 كيف يحدث البحث الدلالي؟

عند وصول سؤال المستخدم، يتحول السؤال إلى Vector بالموديل نفسه. تقارن Chroma هذا الـ Vector بالـ Vectors المخزنة باستخدام cosine similarity، ثم تعيد أفضل النتائج مرتبة من الأقرب للأبعد.

- السؤال يحصل على prefix باسم query.
- الموديل ينتج Vector طوله 384.
- Vector تقارن السؤال بالـ Chroma 33.
- تعود أفضل 3 إلى 5 Chunks مع النص والـ Metadata.
- نستخدم الصفحة ورقم التوصية لعرض المصدر.

Arabic query: follow-up after colorectal cancer surgery
Top result: ng151-rec-1-6-1
Recommendation: 1.6.1 | Page: 26

14 اختبار الاسترجاع

جهزنا خمسة أسئلة بالإنجليزي والعربي. لا يكفي أن يعمل الكود؛ اعتبرنا الاختبار ناجحاً فقط إذا ظهر Chunk متوقع وصحيح ضمن أول خمس نتائج.

- علاج سرطان المستقيم المبكر بالإنجليزي.
- الانسداد الحاد في الجانب الأيسر من الأمعاء الغليظة بالإنجليزي.
- المتابعة بعد الجراحة العلاجية بالإنجليزي.
- علاج سرطان المستقيم المبكر بالعربي.
- المتابعة بعد الجراحة العلاجية بالعربي.

النتيجة النهائية

5 / 5 اختبارات نجحت. سؤال المتابعة بالعربي والإنجليزي أعاد Recommendation 1.6.1 في المركز الأول.

15 ما الذي انتهى وما الخطوة التالية؟

Status	Output	Progress
DONE	Official NICE NG151 PDF	اختيار وتنزيل المصدر
DONE	Markdown + pages JSONL	تحويل الملف
DONE	178 total / 33 MVP	بناء الـ Chunks
DONE	384-d embeddings	إنشاء الـ Embeddings
DONE	Local Chroma DB	التخزين والبحث
DONE	5 / 5 retrieval tests	الاختبار
NEXT	LLM answer + citations	توليد إجابة RAG

إحنا خالصنا إعداد المصدر، التحويل، الـ Chunking، الـ Embeddings، التخزين، والبحث. الخطوة التالية هي RAG Answer Generation: ليكتب إجابة مقيدة بالمصدر LLM ونرسلها مع السؤال إلى Chunks نأخذ أفضل

شكل الإجابة النهائية المتوقع

إجابة واضحة للمستخدم، ثم Recommendation ID ورقم الصفحة ورابط NICE. النموذج لا يفترض معلومات من ذاكرته؛ يستخدم الـ Context المسترجع فقط.

```
User question
-> Embed query
-> Retrieve top chunks from Chroma
-> Send question + context to LLM
-> Answer + recommendation + page + source
```

دليل الملفات والأوامر 16

الملفات التالية تمثل كل مرحلة من المشروع، ويمكن استخدامها في العرض أو عند إعادة تشغيل الـ Pipeline.

```
data/raw/nice_ng151_colorectal_cancer.pdf
data/source_manifest.json
data/processed/nice_ng151_colorectal_cancer.md
data/processed/nice_ng151_colorectal_cancer_pages.jsonl
data/chunks/nice_ng151_colorectal_cancer_chunks.jsonl
data/vector_store/chroma/
data/vector_store/index_report.json
data/evaluation/retrieval_report.json
```

أوامر التشغيل الأساسية:

```
.\.venv\Scripts\python.exe scripts\01_pdf_to_markdown.py
.\.venv\Scripts\python.exe scripts\02_build_chunks.py
.\.venv\Scripts\python.exe scripts\03_build_vector_index.py
.\.venv\Scripts\python.exe scripts\04_test_retrieval.py
```

لتجربة سؤال يدوي:

```
.\.venv\Scripts\python.exe scripts\04_test_retrieval.py --query
"What follow-up is recommended after curative colorectal cancer surgery?"
```

الخلاصة

المشروع الآن يمتلك Knowledge Base طبية صغيرة لكنها منظمة ومختبرة. الخطوة التالية ليست البحث، لأن البحث انتهى ونجح؛ الخطوة التالية هي بناء Prompt وربط نتائج Chroma بالـ LLM لإنتاج إجابة موثقة.